

Project Summary: Multilingual Language Identification

Overview of the Task

Our project focuses on identifying the language of short text snippets drawn from multilingual Wikipedia. Each snippet belongs to one of ten target languages across both Latin and Cyrillic scripts (e.g., English, German, French, Urdu, Kazakh). The goal is to automatically recognize the language of a sentence so downstream systems—search, content moderation, or localization workflows—can route content correctly.

Key Challenges

Several languages share alphabets and common vocabulary, which makes them hard to distinguish at the sentence level. Some inputs are very short, contain names or numbers, or borrow words from other languages, all of which reduce the number of reliable clues. We also worked with multiple scripts (Latin vs. Cyrillic), which can introduce transliteration and encoding edge cases.

External Resources Used

We used the multilingual Wikipedia data provided for the course and processed it with Stanza for sentence segmentation. For modeling, we relied on standard, open-source tools: scikit-learn for the lightweight machine learning baseline, and the Hugging Face Transformers library for the multilingual XLM-R model. These resources allowed us to compare interpretable baselines with a modern transformer approach.

Solution Implemented

We evaluated three approaches and selected the most effective and practical one as our primary solution. First, we built a rule-based heuristic system that checks scripts, diacritics, and hand-curated cue words. Second, we trained a character n-gram logistic regression model, which learns statistical patterns in spelling and is computationally cheap. Third, we fine-tuned the multilingual transformer XLM-R for sequence classification. The character n-gram model delivered the highest accuracy (about 97%) while remaining easy to train and deploy, so it is the recommended choice for the project’s final submission.

Limitations

Despite strong results, the system still struggles when sentences are extremely short, when they contain mostly names or numbers, or when languages share very similar spelling patterns. The n-gram model can also be sensitive to domain

shifts (e.g., moving from Wikipedia to social media) and may misclassify code-switched text that mixes languages in a single sentence. The transformer model can help in these cases but is significantly more expensive to run.

Possible Next Steps

To improve robustness, we could expand training data to include more informal text and code-switching examples. We could add lightweight confidence scoring and a fallback workflow that routes uncertain predictions to a transformer model or human review. Finally, adding more languages would be straightforward for the n-gram model as long as labeled data is available, making the system scalable for broader multilingual support.